Project CelLab – Deliverable 1

Identificarea echipei şi analiza soluțiilor tehnice

Disciplina: Inteligență Artificială

1. Identificarea echipei şi distribuția sarcinilor (Trello)

Membru	Rol / Modul	Responsabilități cheie (Trello)
Curmei Bianca-Gabriela	Motorul de simulare	Implementarea logicii de tranziție a celulelor, stările celulelor, gestionarea matricii NumPy, aplicarea regulilor Game of Life și testarea corectitudinii simulării.
Preutu-Grigore Diana-Smaranda	Interfața grafică	Crearea aplicației web interactive în Streamlit: introducerea descrierilor în limbaj natural de către utilizator, configurarea parametrilor și vizualizarea evoluției grilei.
Dughilă Miruna	Limbaj natural & validare	Preluarea descrierii textuale a utilizatorului, parsarea acesteia, validarea regulilor, gestionarea erorilor și transformarea într-o reprezentare structurată cu reguli și stări.
Ursache David	Analiză, date, experiment AI	Salvarea datelor relevante, determinarea ciclurilor, numărarea celulelor, verificarea stărilor stabile și pregătirea setului de date pentru experimentul final de învățare automată.

2. Scopul proiectului

Scopul proiectului CelLab este dezvoltarea unei aplicații interactive pentru definirea, simularea şi analiza automatelor celulare. Utilizatorul va putea descrie un automat celular în limbaj natural, specificând dimensiunile lumii, stările posibile ale celulelor și regulile de evoluție. Sistemul va interpreta și valida descrierea, va simula generațiile succesive și va permite vizualizarea şi salvarea rezultatelor. De asemenea, aplicația va răspunde la întrebări despre evoluția automatului şi va permite realizarea unui experiment de inteligență artificială pe baza unui set de date obținut din simulări.

3. Soluția tehnică propusă

Pentru implementare propunem utilizarea limbajului Python, datorită suportului său pentru simulări, prelucrarea datelor şi dezvoltarea experimentelor de inteligenţă artificială.

Tehnologiile utilizate sunt:

- **Python:** logica aplicației și motorul de simulare;
- **Streamlit:** dezvoltarea interfeței grafice web interactive;
- **NumPy:** reprezentarea matriceală și prelucrarea eficientă a grilelor (1D și 2D);
- **JSON şi SQLite:** salvarea configuraţiilor, a istoricului generaţiilor și a datelor pentru experimente;
- **scikit-learn:** dezvoltarea şi evaluarea unui model de învăţare automată pe date simulate;
- **pytest:** teste automate pentru verificarea regulilor, a simulării și a analizei;
- **Git şi GitHub:** versionarea codului și colaborarea distribuită a celor patru membri.

Pentru interpretarea limbajului natural se va utiliza inițial un parser bazat pe reguli, care va extrage informațiile necesare din descriere, urmând ca sistemul să poată fi extins pentru a gestiona formulări și descrieri mai complexe.

4. Arhitectura propusă

Aplicația este organizată în următoarele componente principale:

1. **Interfața grafică:** responsabilă de introducerea descrierilor şi configurarea simulărilor.
2. **Modulul de interpretare și validare:** transformă descrierea în limbaj natural într-o reprezentare structurată a automatului.
3. **Motorul de simulare:** calculează generațiile următoare conform regulilor definite.
4. **Modulul de analiză:** răspunde la întrebări despre stări viitoare, stabilitate, configuraţii, numărul celulelor şi cicluri.
5. **Modulul de stocare și experimentare:** păstrează datele relevante şi permite construirea unui set de date pentru un experiment de învăţare automată.

Componentele vor comunica printr-o reprezentare comună a automatului, care va conține dimensiunile lumii, stările celulelor, vecinătatea, regulile de tranziţie şi configuraţia inițială.

5. Arhitectura detaliată pe fluxul de date

- **Introducerea datelor (Interfața grafică):** Utilizatorul introduce descrierea text a automatului și configurează parametrii de simulare direct în interfața web dezvoltată cu Streamlit.
- **Extragerea regulilor (Modulul de interpretare / NLP Python):** Textul este preluat de logica Python, care validează consistența, extrage dimensiunile lumii, stările și regulile, împachetându-le într-un format structurat JSON.
- **Calculul generațiilor (Motorul de simulare NumPy):** Motorul primește fișierul JSON, convertește grila într-o matrice NumPy și aplică simultan regulile de tranziție, calculând rapid generațiile succesive.

- **Afișarea și salvarea (Stocare JSON):** Matricea calculată este trimisă către Streamlit pentru redare vizuală. Concomitent, istoricul fiecărei generații simulate este salvat continuu în format JSON/SQLite pentru acces ulterior.
- **Interogarea și experimentul AI (Analiză și ML):** Modulul de analiză parcurge istoricul salvat pentru a răspunde matematic la întrebări, iar datele agregate sunt introduse în scikit-learn pentru antrenarea și evaluarea modelului predictiv.

6. Automate celulare propuse (1D și 2D)

Implementarea inițială utilizează **Game of Life (2D)**, un automat celular bidimensional cu două stări, în care evoluția celulelor depinde de numărul vecinilor vii. Suplimentar, proiectul include implementarea automatului **Brian's Brain (2D)**, care utilizează trei stări (activă, refractară, inactivă), precum și a automatului unidimensional **Rule 30 (1D)**.

Aceste exemple permit testarea unor tipuri diferite de reguli şi evaluarea capacității sistemului de a gestiona automate celulare distincte.

7. Funcționalități planificate

Aplicația urmărește să ofere următoarele funcționalități:

- Introducerea și validarea descrierilor în limbaj natural (cu semnalarea formulărilor incomplete sau contradictorii);
- Configurarea manuală sau generarea automată a lumii inițiale;
- Simularea pas cu pas sau continuă a evoluției automatului pe un număr ales de generații;
- Vizualizarea interactivă a grilei și navigarea între generațiile simulate;
- Salvarea și exportul configurațiilor și al istoricului stărilor;
- Analiza automată a rezultatelor și răspunsul la întrebări specifice;
- Realizarea unui experiment de inteligență artificială pe baza unui set de date obținut din simulări.

8. Specificații funcționale obligatorii

Componentă	Ce vom implementa
Descrierea automatului	Introducerea și interpretarea regulilor exprimate în limbaj natural
Validare	Detectarea descrierilor incomplete, ambigue sau contradictorii
Motor de simulare	Calcularea generațiilor succesive pe grile 1D și 2D folosind NumPy
Lume inițială	Configurare manuală sau generare automată procedurală
Interfață grafică	Afișarea grilei și navigarea între generații în Streamlit

9. Cele 5 întrebări la care răspunde aplicația

1. **Care este starea după N generaţii?** Se simulează exact N pași și se returnează grila rezultată.
2. **Când ajunge sistemul într-o stare stabilă?** Se compară generațiile consecutive; dacă două stări consecutive sunt identice, s-a atins o stare stabilă.
3. **Apare vreodată o anumită configuraţie?** Se caută configuraţia în generațiile simulate.
4. **Câte celule de tip X există după N generaţii?** Se numără celulele cu starea cerută în generaţia N.
5. **Intră simularea într-un ciclu?** Se memorează stările anterioare; dacă o stare se repetă, se identifică perioada ciclului repetitiv.

10. Experimentul de Inteligență Artificială

> Experimentul de învățare automată constă în colectarea datelor simulate prin rularea automatelor celulare, extragerea caracteristicilor relevante din traiectoriile generațiilor, antrenarea unui model predictiv cu **scikit-learn** și evaluarea performanței acestuia pe un set dedicat de date de test.